

2026-06-19-ota-management-client-design

Helix OTA — Cross-Platform Management Client Design

Revision: 1 Last modified: 2026-06-19T01:00:00Z Status: APPROVED

1. Architecture Overview

Stack: Tauri v2 (Rust) + React 19 + TypeScript 5 + Shadcn UI + Tailwind CSS 4 + TanStack Query 5 **Deployment:** Containerized via vasic-digital/containers submodule (Podman/Docker) **Code sharing:** ~95% across platforms

Platform Strategy

Platform	Shell	UI Layer	Binary	RAM
Web	Browser (served by ota-server)	React SPA	~500KB JS	~30MB
Desktop (macOS/ Win/Linux)	Tauri v2 (Rust)	React via WebView	~5MB	~30MB
Mobile (iOS/ Android)	Tauri Mobile	React via WebView	~8MB	~40MB

The SAME React SPA runs everywhere. Tauri Rust layer provides native capabilities: - USB device flashing - Filesystem access for image files - System tray for background monitoring - Push notifications on deployments

2. Technology Stack

Frontend

- **React 19** — Concurrent features, Server Components (web), Suspense
- **TypeScript 5** — Strict mode, path aliases
- **Shadcn UI** — Accessible, composable component library

- **Tailwind CSS 4** — Utility-first CSS, CSS-first configuration
- **TanStack Query 5** — Server state, caching, optimistic updates
- **TanStack Router** — Type-safe routing, nested layouts
- **Zustand** — Lightweight client state (preferences, UI state)
- **React Hook Form + Zod** — Form validation

Desktop Shell (Tauri)

- **Rust** — Native plugins for USB, tray, file dialogs
- **tauri-plugin-shell** — Sidecar management
- **tauri-plugin-fs** — File system access for images
- **tauri-plugin-notification** — Deployment alerts

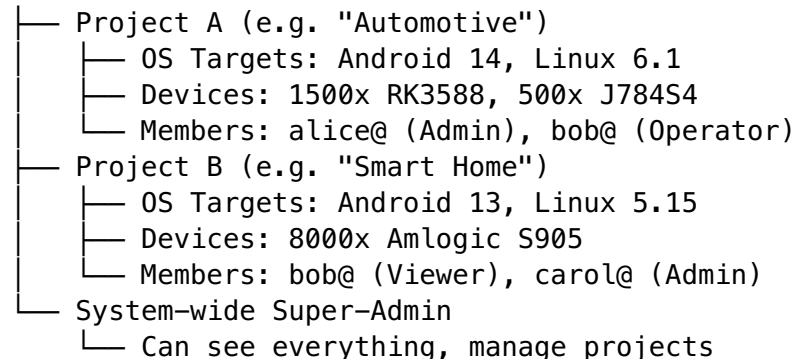
Build Tooling

- **Vite** — Dev server, bundling, HMR
- **pnpm** — Fast, disk-efficient package management
- **Tauri CLI** — Cross-platform build, code signing

3. Multi-Project & Permissions Model

Project Hierarchy

Helix OTA Root



Permission Dimensions

```

User Permissions {
  super_admin: bool, // Bypasses all checks

  projects: {
    [project_id]: {
      role: "admin" | "operator" | "viewer", // Project-level role
      os_types: ["android", "linux"], // Which OS types
      targets: ["rk3588", "s905"], // Which hardware targets
    }
  }
}
  
```

```
}
}
```

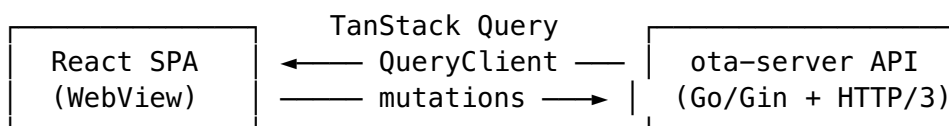
UI Behavior by Role

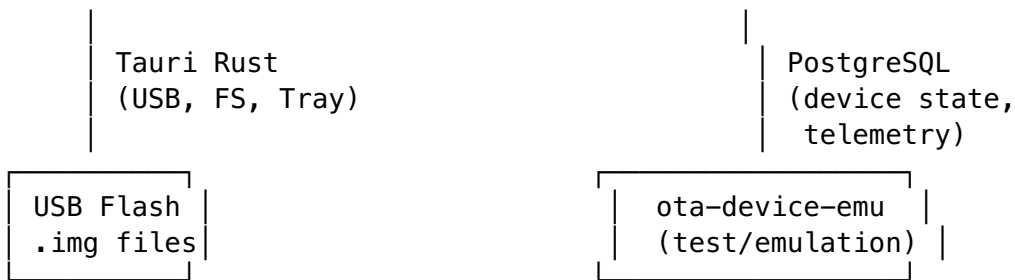
Feature	Viewer	Operator	Admin	Super-Admin
View dashboards	✓	✓	✓	✓
View device details	✓	✓	✓	✓
View audit log	✓	✓	✓	✓
Create/update releases	✗	✓	✓	✓
Create rollouts	✗	✓	✓	✓
Manage projects	✗	✓	✓	✓
Manage users	✗	✗	✓	✓
System settings	✗	✗	✗	✓
Delete resources	✗	✗	✓	✓

4. Application Routes

/login	– Auth (login/register)
/dashboard	– Project overview dashboard
/projects	– Project list
/projects/:id	– Single project detail
/projects/:id/settings	– Project settings
/devices	– Fleet device list
/devices/:id	– Single device detail
/devices/:id/telemetry	– Device telemetry history
/images	– Image/artifact repository
/images/:id	– Single image detail
/releases	– Release list
/releases/:id	– Release detail
/deployments	– Deployment list
/deployments/:id	– Deployment detail
/deployments/:id/rollout	– Rollout control
/deployments/:id/recall	– Recall/rollback
/groups	– Device groups
/groups/:id	– Group detail
/audit	– Audit log
/users	– User management (admin)
/settings	– System settings

5. Data Flow

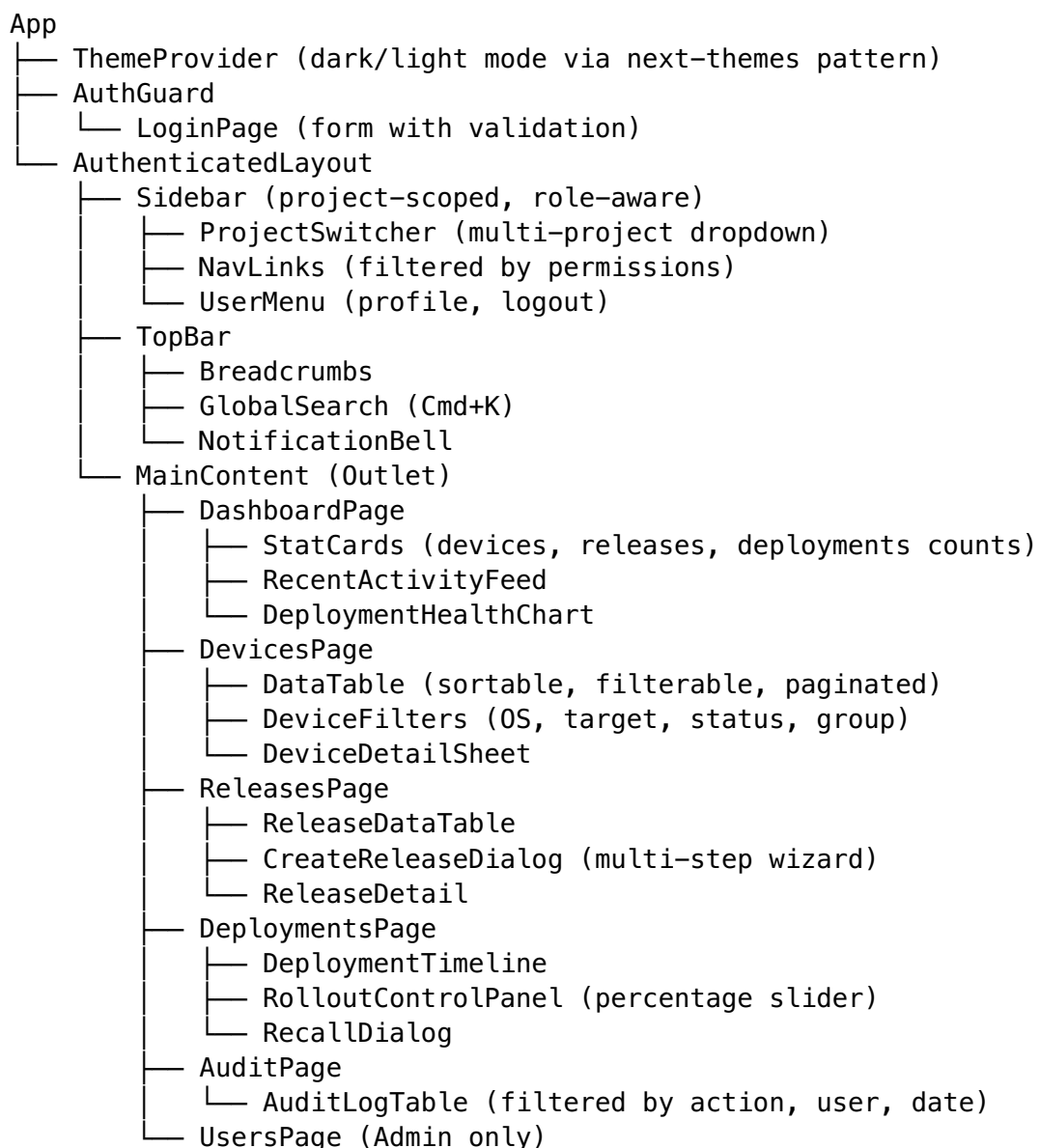




Real-time Updates

- TanStack Query `refetchInterval` for polling (configurable: 5s dashboard, 30s device list)
- Optional: Server-Sent Events (SSE) for push updates on deployment status changes
- Tauri notification plugin for deployment completion alerts

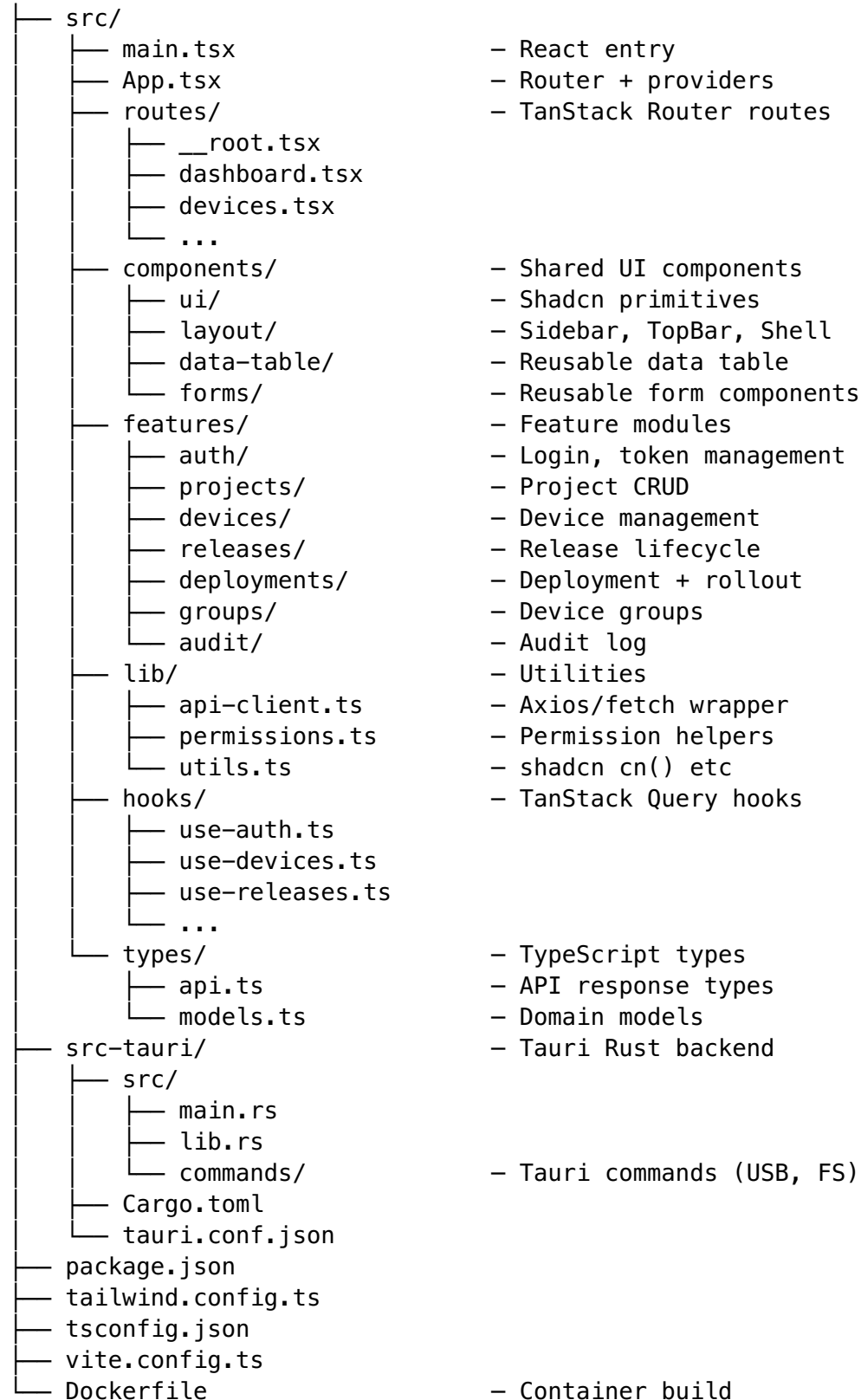
6. UI Component Tree (Shadcn-based)



- └─ UserTable
- └─ PermissionEditor (multi-dimensional matrix)

7. Project Structure

clients/ota-manager/



8. Containerization

All components run in containers via the containers submodule:

```
containers/ota-manager/  
├─ Dockerfile                - Multi-stage: node build →  
nginx + static  
├─ docker-compose.yml        - ota-manager + ota-server +  
postgres  
└─ podman-compose.yml        - Same for Podman
```

The web client is built as a static SPA and served by: - **Development**: Vite dev server (hot reload, proxied to ota-server) - **Production**: Nginx container serving built assets, reverse-proxied to ota-server - **Desktop**: Tauri embeds the same SPA in a WebView, connects directly to ota-server API

9. Testing Strategy

Layer	Tool	What
Unit (components)	Vitest + Testing Library	Individual component behavior
Unit (hooks)	Vitest + MSW	TanStack Query hook tests with mocked API
Integration	Playwright	Full page flows (login → dashboard → deploy)
E2E	Playwright + Tauri e2e	Desktop-specific features (USB flash, tray)
Visual	Chromatic / Storybook	Component visual regression
API contract	Zod schemas validated against server	TypeScript types match API responses

10. Build & Release Pipeline

```
# Development  
cd clients/ota-manager  
pnpm install  
pnpm dev                # Vite dev server (web only)  
pnpm tauri dev          # Tauri desktop app  
  
# Build web  
pnpm build              # Static output to dist/  
  
# Build desktop  
pnpm tauri build        # .dmg / .msi / .AppImage
```

```
# Container build
podman build -t ota-manager:latest .
podman-compose up      # Full stack: manager + server + db

# Mobile
pnpm tauri build android  # .apk / .aab
pnpm tauri build ios      # .ipa
```

11. Design Assets

- Shadcn UI components: dialog, dropdown, data-table, form, sheet, tabs, toast, skeleton, badge, card, progress
 - Icons: Lucide (consistent, MIT-licensed, bundled with Shadcn)
 - Theme: Dark/light mode via CSS variables, persists in Zustand
 - Layout: Sidebar navigation (collapsible), top breadcrumbs, content area
-

Spec Self-Review

- ☒ No TBD/TODO placeholders
- ☒ Architecture matches feature descriptions
- ☒ Scope is focused on a single management client (not trying to build the whole platform)
- ☒ Requirements are unambiguous and actionable
- ☒ References existing server API (35+ endpoints) and containers submodule
- ☒ Testing strategy covers all layers
- ☒ Containerization plan is concrete

Next: Implementation plan → parallel subagent execution.